Hardware Design for AI-Course Project

Himadri Das – IIT Jodhpur – MTech Final Year

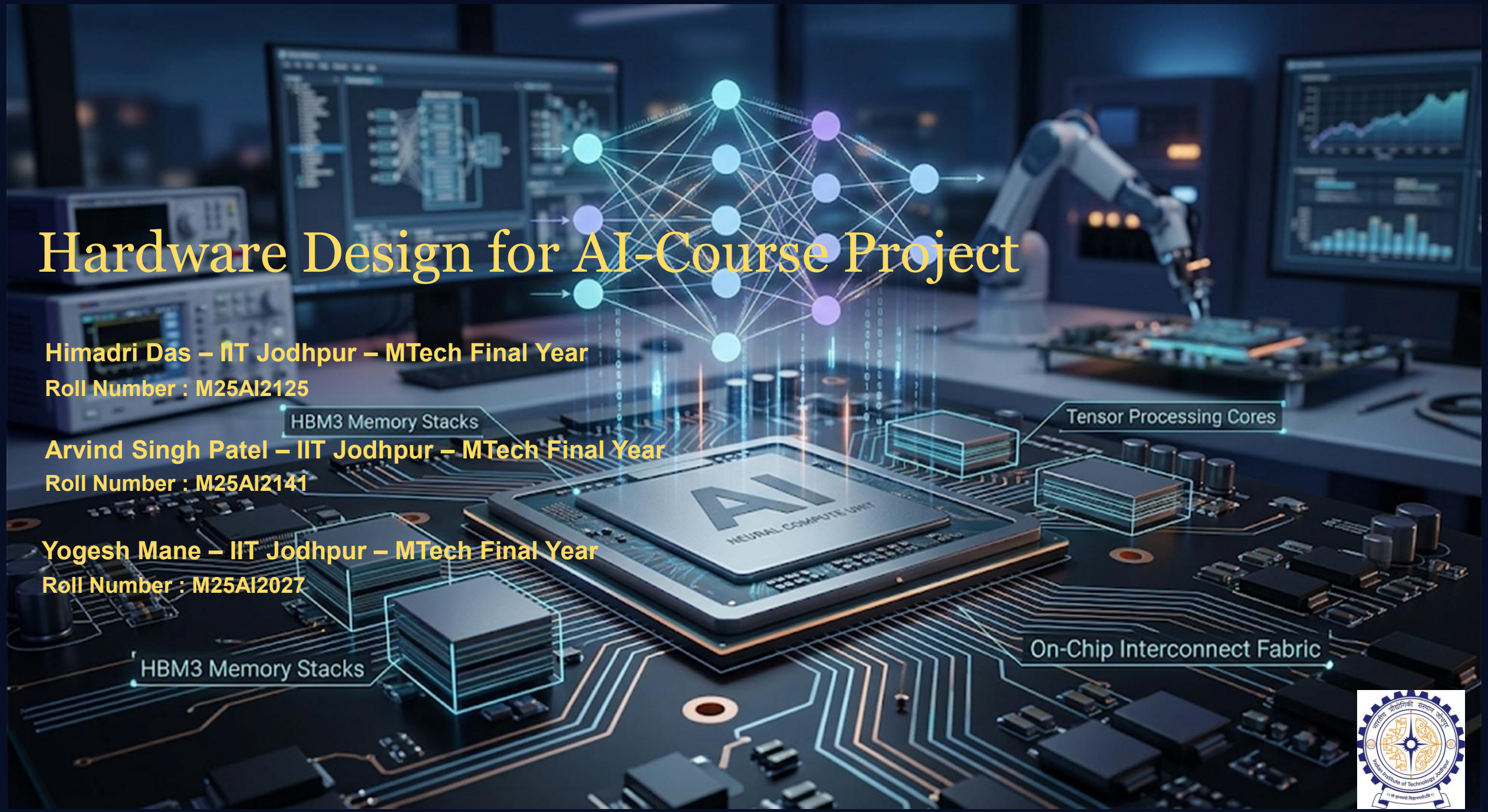
Roll Number : M25AI2125

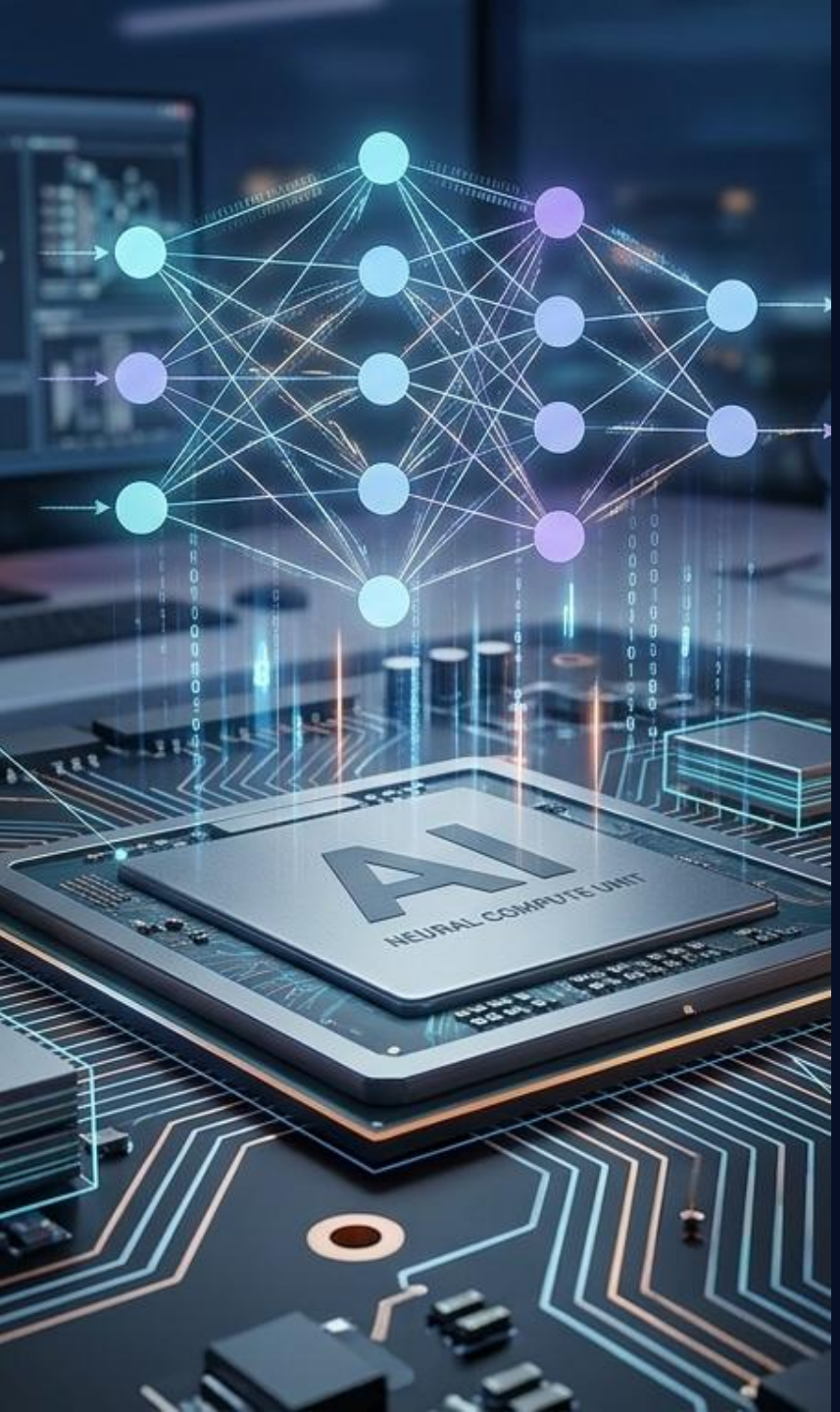
Arvind Singh Patel – IIT Jodhpur – MTech Final Year

Roll Number : M25AI2141

Yogesh Mane – IIT Jodhpur – MTech Final Year

Roll Number : M25AI2027





B12-Neural Network Optimization using Apache TVM

01. Assignment Topic :

- **Optimize Operator Scheduling**
- **Benchmark Performance**

02. Indicative Approach :

- **Model Translation & Compilation**
- **Scheduling & Performance Validation**

03. Reports :

- **Abstract**
- **Solution Approach**
- **Implementation & Execution**
- **Observations**
- **Reports & Analyse**

By - Himadri Das (M25AI2125), Arvind Singh Patel (M25AI2141) & Yogesh Mane (M25AI2027)

Overview

Abstract

This report presents an end-to-end framework for optimizing pretrained neural networks using Apache TVM. The objective is to map high-level models to optimized hardware kernels using operator scheduling and to evaluate performance improvements via latency and throughput metrics. Results demonstrate significant speedup without retraining

Objective

- Optimize neural network execution
- Apply operator scheduling
- Benchmark performance
- Analyze hardware efficiency

Executive Summary

Introduction

Deep learning models are computationally expensive and often inefficient on general-purpose hardware. Apache TVM addresses this by compiling models into optimized kernels tailored to the target hardware.

Problem Statement

Improve inference performance of pretrained neural networks without modifying model architecture using compiler-level optimization.

What Aiming to Achieve

Make AI models run faster using smart optimization without changing the model.

Kee Needs

Improve AI model performance using hardware-aware optimization.

GitHub Repos

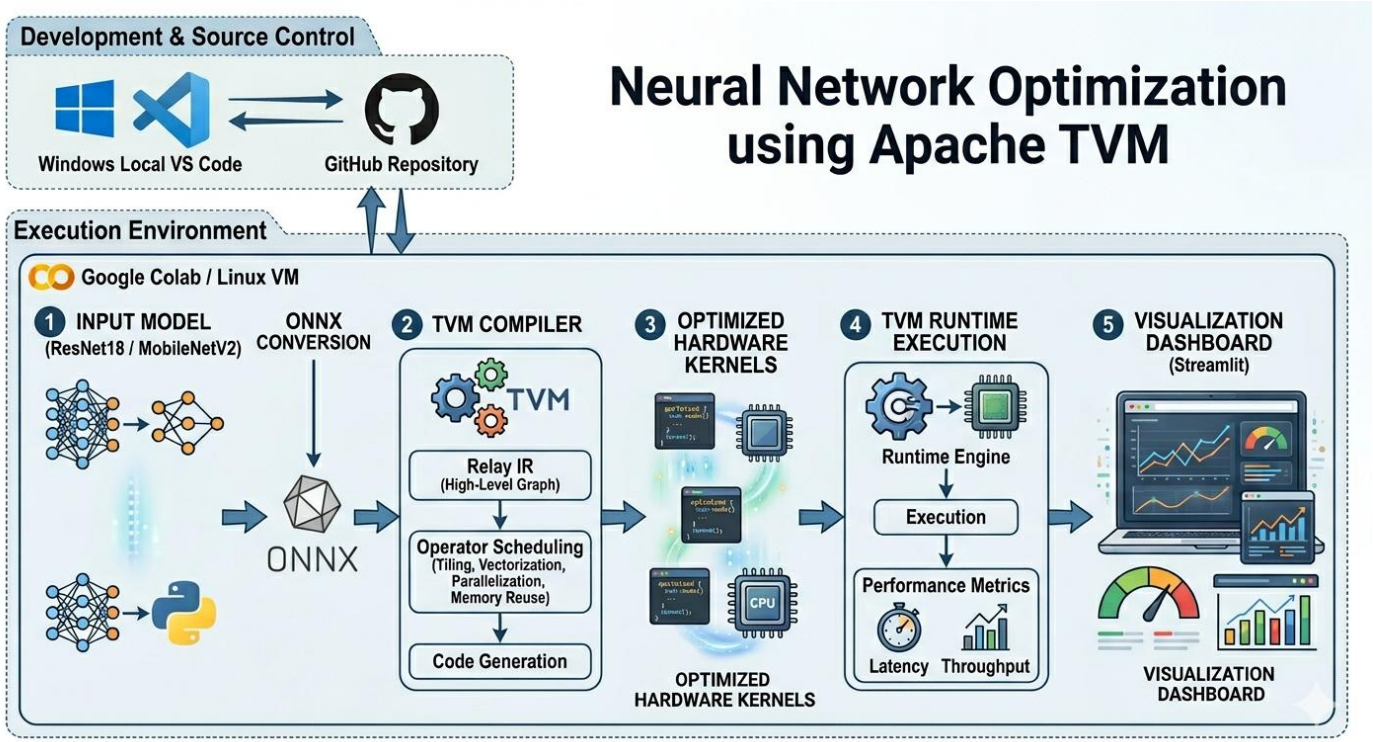
https://github.com/m25ai2125-glitch/HD-Course-Project.git

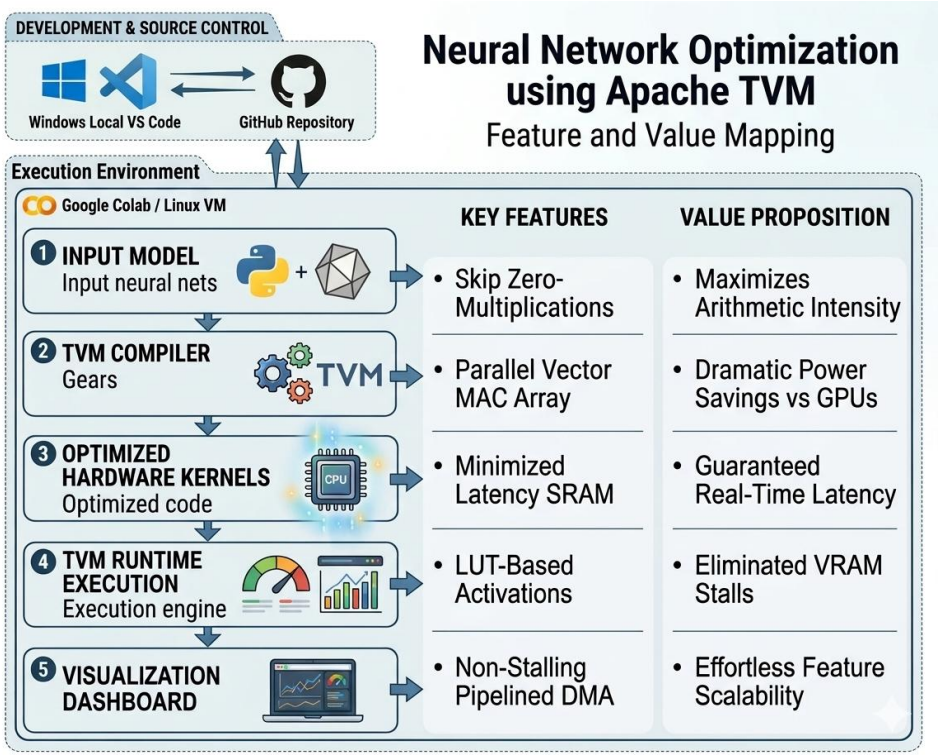
Demo Presentation Video

https://drive.google.com/file/d/1V2JwILVUmBb2yvfZYS_2y_AZLcdAdW6i/view?usp=drive_link

By - Himadri Das (M25AI2125), Arvind Singh Patel (M25AI2141) & Yogesh Mane (M25AI2027)

Solution Approach - Architecture





Methodology in Step-By-Step:

1. Load pretrained model
2. Export to ONNX
3. Baseline benchmark
4. TVM compilation
5. Apply scheduling optimizations
6. Execute optimized model
7. Compare results

Operator Scheduling:

Operator scheduling optimizes execution using:

- Loop tiling
- Parallelization
- Vectorization
- Memory reuse

Mathematically:

Total Execution Time ≈ Compute Time + Memory Access Time

Optimization reduces memory bottlenecks and increases compute efficiency

Hardware Optimization Concepts:

Key factors:

- Arithmetic Intensity = FLOPs / Memory Access
- Cache Utilization
- Parallel Execution Units
- Memory Bandwidth

TVM improves arithmetic intensity by maximizing computation reuse.

Implementation Details

Execution→ The project is implemented using Python in Google Colab and Linux environments. Key libraries include PyTorch, ONNX, ONNX Runtime, and Apache TVM. TVM is built from source and configured for CPU/GPU execution.

Tools Used

- PyTorch
- ONNX
- ONNX Runtime
- Apache TVM

Execution Environment:

- Google Colab / Linux VM
- Widows VS Code (Local)
- GitHub
- Stream lite Cloud

DR/POC/Fallback

- Google Cloud VMs
- Multipass
- WSL (Local)
- Powershell

Results Table

Batch	Baseline (ms)	TVM (ms) (CPU)	Speedup	Throughput
1	69.459407	289.307240	0.24 x	3.46
4	260.003896	1757.678767	0.15 x	2.28
8	563.535473	3457.544690	0.16 x	2.31

Performance Analysis → The results show that TVM execution is slower than the baseline **ONNX** Runtime. This is because advanced scheduling techniques such as MetaSchedule tuning were not applied. Additionally, execution was performed **on CPU** without hardware-specific optimizations

By - Himadri Das (M25AI2125), Arvind Singh Patel (M25AI2141) & Yogesh Mane (M25AI2027)

Challenges & Limitations

Challenges Faced

- **Complex TVM Installation:** Installing Apache TVM is difficult as it requires building from source with dependencies like LLVM, CMake, and compilers.
- **Environment Compatibility Issues :**TVM works best with specific Python versions (3.10/3.11), causing issues in environments like Google Colab or Windows.
- **Disk Space Constraints:** Building TVM requires significant storage, and limited disk space can lead to build failures.
- **Difficult Setup of Dependencies:** Proper configuration of tvm-ffi, TVM_LIBRARY_PATH, and Python bindings is error-prone.
- **Multi-Environment Execution:** Using Windows, Linux (VM/Multipass), and Colab together creates path and execution inconsistencies.
- **Understanding Operator Scheduling:** Learning low-level optimization concepts such as scheduling, parallelization, and memory reuse is challenging.
- **Benchmarking Complexity :** Ensuring fair comparison between baseline and optimized execution requires careful setup and consistent parameters.

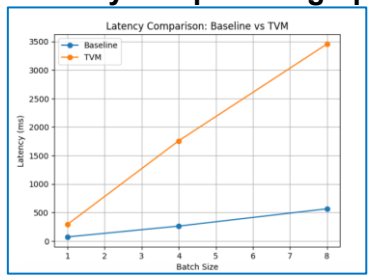
Limitations of the Current Work

- **No Advanced Scheduling (MetaSchedule Not Used):** The current implementation uses default compilation without auto-tuning, leading to suboptimal performance.
- **CPU-Only Execution:** The project does not utilize GPU acceleration, limiting performance improvements.
- **TVM Underperformance Compared to Baseline:** Results show that TVM is slower than ONNX Runtime due to lack of tuning and hardware-specific optimization.
- **Limited Optimization Scope:** Only basic optimization was applied; advanced techniques like kernel fusion and auto-scheduling were not fully explored.
- **No Real-Time Deployment:** The model was not deployed in a real-world or edge environment.
- **Static Batch Processing:** The system does not dynamically adapt to different workloads or input sizes.

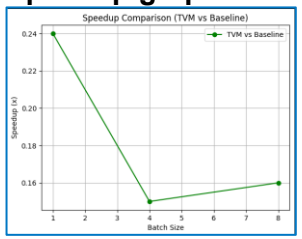
Overall Conclusion → The project highlights that while Apache TVM provides powerful optimization capabilities, achieving performance gains requires proper scheduling, tuning, and hardware-aware configuration.

Reports & Analyses - Charts & Screenshots

Latency comparison graph



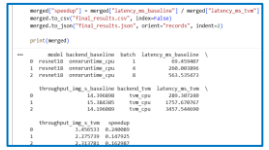
Speedup graph

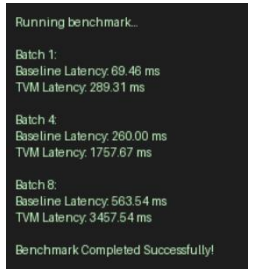


Benchmark output tables

Model	Batch	Backend (Baseline)	Baseline Latency (ms)	Through put (img/s)	Backend (TVM)	TVM Latency (ms)	TVM Throughput (img/s)	Speedup
ResNet18	1	ONNX Runtime (CPU)	69.46	14.4	TVM (CPU)	289.31	3.46	0.24×
ResNet18	4	ONNX Runtime (CPU)	260	15.38	TVM (CPU)	1757.67	2.28	0.15×
ResNet18	8	ONNX Runtime (CPU)	563.54	14.2	TVM (CPU)	3457.54	2.31	0.16×

Execution screenshots





Latency comparison graph : The results highlight that compiler-based optimization requires proper tuning mechanisms such as MetaSchedule to achieve performance gains.

Speedup Graph: The results emphasize that without proper operator scheduling and auto-tuning, compiler-based optimization may not outperform highly optimized runtime frameworks.

Benchmark Output Tables: The results highlight that default compilation without scheduling optimization is insufficient, reinforcing the importance of hardware-aware tuning in ML compilers.

Execution Screenshots: Execution output showing latency comparison between baseline ONNX Runtime and TVM optimized model across different batch sizes.

Key Observation

Speedup values are less than 1, indicating performance degradation. This highlights that compiler optimization alone is not sufficient without proper scheduling and tuning

Conclusion

The experiment demonstrates the importance of operator scheduling and hardware-aware optimization. Future improvements can significantly enhance performance..

Future Work

- MetaSchedule tuning
- GPU optimization
- Edge deployment
- Real-time systems

*****These improvements are expected to transform the current system from a baseline compiler implementation into a fully optimized, hardware-aware AI execution pipeline.**

Key Learnings

- **Understanding ML Compilers (TVM):** Learned how Apache TVM acts as a compiler that converts high-level neural networks into optimized low-level hardware execution.
- **Model-to-Hardware Mapping :** Gained insight into how deep learning models are mapped onto CPU/GPU architectures for efficient execution.
- **Importance of Operator Scheduling :** Understood that performance improvement depends heavily on scheduling techniques such as loop tiling, parallelization, and memory optimization.
- **Performance Benchmarking Skills :** Learned how to measure and compare latency, throughput, and speedup between baseline and optimized models.
- **Optimization ≠ Automatic Speedup :** Realized that simply compiling with TVM does not guarantee better performance without proper tuning.
- **Role of Hardware Awareness :** Learned that optimization must consider hardware factors like memory bandwidth, cache usage, and parallel processing capability.
- **Working with ONNX Pipeline :** Understood the process of converting models to ONNX format for interoperability between frameworks.
- **Debugging Complex Environments :** Gained experience handling installation issues, environment setup, and dependency management across multiple platforms.
- **Limitations of Default Execution :** Observed that ONNX Runtime can outperform untuned TVM due to built-in optimizations.
- **Need for Advanced Optimization Techniques :** Learned the importance of advanced methods like MetaSchedule, auto-tuning, and GPU acceleration for achieving real performance gains.

- This project demonstrates that hardware-aware optimization using ML compilers like TVM is essential for improving neural network performance.
- It also highlights that effective speedup requires proper operator scheduling, tuning, and alignment with underlying hardware capabilities.

Reference :

1. Apache TVM Docs
2. ONNX Runtime Docs
3. PyTorch Docs
4. Research Papers

By - Himadri Das (M25AI2125), Arvind Singh Patel (M25AI2141) & Yogesh Mane (M25AI2027)

Thank you

Himadri Das (M25AI2125) , Arvind Singh Patel (M25AI2141) & Yogesh Mane (M25AI2027)